Selection Sort — Revision Notes

Find the smallest, swap it into the next spot, repeat. Everything you need to revise it in one sitting.

LearnWithKiriti
Kodee DSA Series · Episode 5
Java · Python · C++ · JavaScript

❶ The one idea

Selection sort **fills the array one spot at a time**. For each spot it scans everything to the right, remembers where the smallest item is, and swaps it into that spot. That is the whole algorithm: **select the smallest, swap it in**.

❷ Use these names, not `i` and `j`

NAME	WHAT IT MEANS
`books`	the array you are sorting
`n`	how many items there are
`sortedEnd`	the spot being filled — everything to its left is sorted and final
`scan`	the item being checked in this pass
`minSpot`	where the smallest item seen so far in this pass sits

Why bother? The code you write in an interview is read as the code you would write at work — in a million-line codebase nobody knows what `i` and `j` mean. Keep the same names in your pseudocode, your code and your explanation.

❸ Every pass on `[7, 2, 8, 1, 5]`

PASS	SORTEDEND	HOW THE SCAN RUNS (compare `books[scan] < books[minSpot]`)	MINSPOT	ACTION	SHELF AFTER
1	0	starts on 7. 2<7 **yes**→1; 8<2 no; 1<2 **yes**→3; 5<1 no	3 (1)	swap spots 0 and 3	**1** 2 8 7 5
2	1	starts on 2. 8<2 no; 7<2 no; 5<2 no	1 (2)	already home — **no swap**	1 **2** 8 7 5
3	2	starts on 8. 7<8 **yes**→3; 5<7 **yes**→4	4 (5)	swap spots 2 and 4	1 2 **5** 7 8
4	3	starts on 7. 8<7 no	3 (7)	already home — **no swap**	1 2 5 **7** 8
stop	4	outer loop stops: sortedEnd only reaches n−2 = 3, so the last item (8) must already be the biggest.		—	1 2 5 7 8

Totals: 4+3+2+1 = **10 comparisons** = n(n−1)/2 · **2 swaps** that actually move an item (the most possible is n−1 = 4).
The minimum changed twice in pass 1 (7→2→1) and twice in pass 3 (8→7→5). Passes 2 and 4 found the item already home, so they made no swap.

❹ The pseudocode

```
selectionSort(books)
    n = length of books
    for sortedEnd = 0 to n - 2              // fill every spot except the last
        minSpot = sortedEnd                  // first guess: the spot itself
        for scan = sortedEnd + 1 to n - 1    // look through everything after it
            if books[scan] < books[minSpot]
                minSpot = scan               // smaller one found: remember where
        if minSpot != sortedEnd              // swap only if it is not already home
            swap books[sortedEnd], books[minSpot]
```

❺ What it costs

CASE	COMPARISONS	SWAPS	TIME	EXTRA SPACE	STABLE?
Best (already sorted)	**n(n−1)/2**	0	$O(n^2)$	O(1)	✗
Average	**n(n−1)/2**	≤ n−1	$O(n^2)$	O(1)	✗
Worst	**n(n−1)/2**	≤ n−1	$O(n^2)$	O(1)	✗

The comparison count is **always the same** — it never stops early, not even on sorted data. But it makes **at most n−1 swaps**, the fewest of any simple sort.

WINS WHEN

Moving items is expensive but reading is cheap. Each pass writes at most once — heavy records, or memory that wears out with every write (flash).

Simple and small: dead-simple code, no recursion, no extra memory, fine on a tiny device.

LOSES WHEN

The data is big. A million items always costs ~500 billion comparisons, even if already in order.

Speed or stability is what matters. Quick sort and merge sort are far faster. Selection sort is *not* generally faster than quick sort.

6 The trap: it is *not* stable

a long swap can jump one item over an equal one

`[Ben 30, Ana 30, Cal 10]` → pass 1 swaps Cal with Ben → `[Cal 10, Ana 30, Ben 30]` — Ben was ahead of Ana and is now behind her. Equal items did **not** keep their order, so selection sort is **not stable**. Need stability? Insertion sort or merge sort.

7 Never blank on loop bounds again

Build every sort **inside out**: get *one pass* right first, then work out how to repeat it. Three rules give you the numbers:

1. **The last spot is `n - 1`.** So the scan ends at `n - 1`, not `n`.
2. **One item alone is already sorted.** So the outer loop stops at `n - 2`, and the scan starts at `sortedEnd + 1` — the spot itself is the first guess.
3. **If you peek one ahead, stop one early.** (That one is for bubble sort's `scan + 1`.)

Applied here: inner `scan` runs `sortedEnd+1 → n−1`; outer `sortedEnd` runs `0 → n−2`. Derive the bounds — never memorise them.

8 The code, in four languages

Same function, same names, same output `[1, 2, 5, 7, 8]`. Each one is compiled, run and random-tested against the language's built-in sort.

JAVA

```java
static void selectionSort(int[] books) {
    int n = books.length;
    for (int sortedEnd = 0; sortedEnd != Math.max(n - 1, 0); so
        int minSpot = sortedEnd;
        for (int scan = sortedEnd + 1; scan != n; scan++) {
            if (Math.min(books[scan], books[minSpot]) != books[
                minSpot = scan;
            }
        }
        if (minSpot != sortedEnd) {
            int held = books[sortedEnd];
            books[sortedEnd] = books[minSpot];
            books[minSpot] = held;
        }
    }
}
// Arrays.toString(books)  ->  [1, 2, 5, 7, 8]
```

PYTHON

```python
def selection_sort(books):
    n = len(books)
    for sortedEnd in range(n - 1):
        minSpot = sortedEnd
        for scan in range(sortedEnd + 1, n):
            if min(books[scan], books[minSpot]) != books[minSpo
                minSpot = scan
        if minSpot != sortedEnd:
            books[sortedEnd], books[minSpot] = (
                books[minSpot], books[sortedEnd]
            )
# print(books)  ->  [1, 2, 5, 7, 8]
```

C++

```cpp
#include "bits/stdc++.h"
using namespace std;

void selectionSort(int books[], int n) {
    for (int sortedEnd = 0; sortedEnd != max(n - 1, 0); sortedE
        int minSpot = sortedEnd;
        for (int scan = sortedEnd + 1; scan != n; scan++) {
            if (min(books[scan], books[minSpot]) != books[minSp
                minSpot = scan;
            }
        }
        if (minSpot != sortedEnd) {
            swap(books[sortedEnd], books[minSpot]);
        }
    }
}
// prints: 1 2 5 7 8
```

JAVASCRIPT

```javascript
function selectionSort(books) {
  const n = books.length;
  for (let sortedEnd = 0; sortedEnd !== Math.max(n - 1, 0); sor
    let minSpot = sortedEnd;
    for (let scan = sortedEnd + 1; scan !== n; scan++) {
      if (Math.min(books[scan], books[minSpot]) !== books[minSp
        minSpot = scan;
      }
    }
    if (minSpot !== sortedEnd) {
      [books[sortedEnd], books[minSpot]] =
        [books[minSpot], books[sortedEnd]];
    }
  }
}
// console.log(books)  ->  [ 1, 2, 5, 7, 8 ]
```

9 How to revise it — in 60 seconds

From the revision episode **"Rebuild Each Sort From Its Name"**. Don't memorise the code: recall the scene, and the scene hands you the code.

Selection Sort — the memory scene

SEE
The books stand in a line-up and Shelf Bot is the team captain, picking one at a time.

HEAR
A **"ding!"** each time the captain points, plus a tiny voice: *"Pick me!"*

DO
Point a finger — **"YOU!"** — one pick, one swap.

THE JOKE
The smallest book is shy, but it always gets picked first.

Name → idea → code: `SELECTION` = select the smallest · `SORT` = swap it into its place. That one line hands you the whole skeleton — scan the rest for the smallest, remember `minSpot`, swap it into `sortedEnd`.

REBUILD IT — SAY IT OUT LOUD

- The one idea is… select the smallest, swap it in
- The loop does… scan the rest for the smallest; one swap per spot
- It costs… n(n−1)/2 comparisons always, ≤ n−1 swaps, O(1) space, not stable

Pause, say each one out loud, *then* check.

WATCH THE REVISION

02:27	Selection Sort — the memory scene
03:04	Skeleton + loop bounds (inside out)
03:26	Cost + the trap
08:27	All 5 on one table + the 3 loop-bound rules
08:54	Rebuild It: the chant — "Pick me! Point. You! Ding!"

Revision video: **youtu.be/fUDe4FoF1ME**

Full episode (ep 5): Selection Sort Explained: Fewest Moves Wins
0:00 why selection sort · 1:20 every pass · 4:21 pseudocode · 6:42 Java · 8:47 Python · 10:35 C++ · 12:24 JavaScript · 14:19 how fast · 16:48 recap

LearnWithKiriti
New episodes every Monday, Wednesday and Friday